AI-Native Transceiver Design for Next-Generation Wireless Networks

Yuxing Mao and Haikun Xu | Supervisor: Dr Huynh Nguyen

Trustworthy Computing, School of Computer Science and Informatics · LivSURF 436026

Notebook and measured data

1 Background and Motivation

In a **conventional transceiver design**, coding, modulation, channel estimation and detection are designed as separate blocks. Each block is optimal for its own task. The complete receiver is limited by the assumptions that link them. Deep learning offers two alternatives: train transmitter and receiver together as an **autoencoder** [1], or replace channel estimation and detection with a **learned receiver** [2].

Research questions:

- How much performance gain does end-to-end learning offer over a **well-optimised classical receiver**?
- Can the transmitter be trained **without a differentiable channel model**?

2 Objectives and Contributions

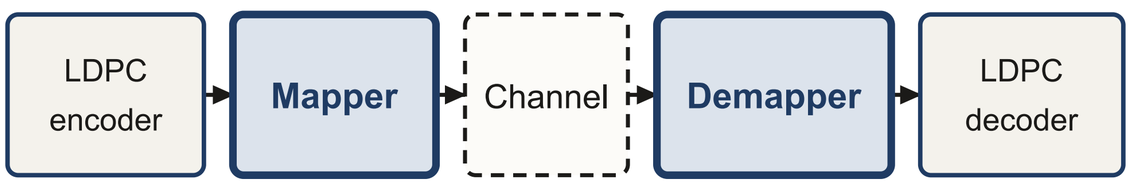
Objectives

- Compare an end-to-end learned transceiver with a **matched** classical system over AWGN and Rayleigh block-fading channels with **pilot-estimated CSI**.
- Evaluate transmitter training by reinforcement learning when **no differentiable channel model** is available.

Contributions

- Develop a fair evaluation that includes **pilot overhead** and **channel-estimation error**.
- Show that reinforcement learning can train the transmitter with only a small performance penalty compared with gradient-based training.
- Show that learning provides a small improvement over 64-QAM on AWGN, mainly through constellation shaping, but **no consistent improvement** under Rayleigh fading.

3 System Model



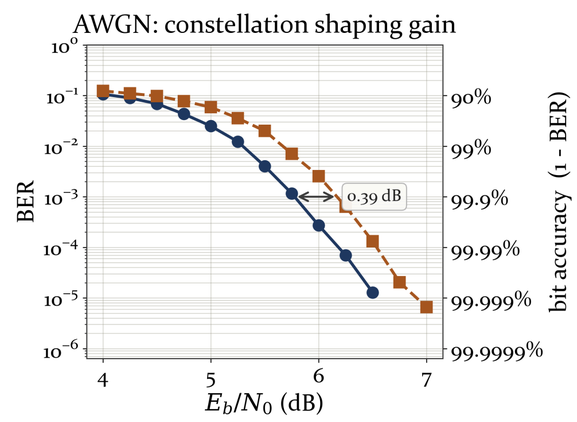
Learned	trainable constellation	neural network
Classical	64-QAM, Gray	APP demapper

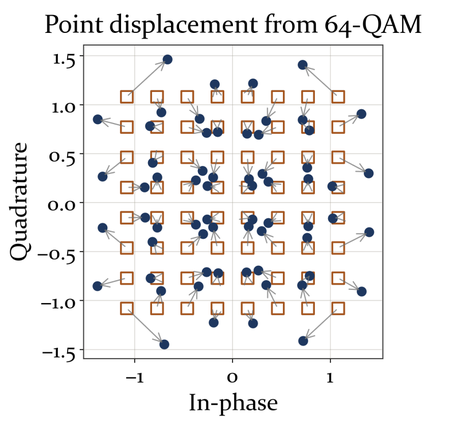
Channel: AWGN or Rayleigh block fading, L = 25, P pilot symbols, LS estimate

The two systems differ only in the shaded blocks. They share the rate-1/2 5G LDPC code, the channel model and the random seeds, so any difference in bit error rate comes from the mapper and the demapper.

Let **R** be the code rate, **L** the number of symbols in one fading block, **P** the number of pilot symbols in that block, $\mathbf{N_0}$ the noise power spectral density and $\hat{\boldsymbol{h}}$ the channel coefficient estimated from the pilots. Both systems are charged the same way: the effective code rate is **R(L−P)/L**, and after zero-forcing the effective noise is $\mathbf{N_0(1+1/P)/|\hat{h}|^2}$.

4 Where the AWGN Gain Comes From





Learned (trainable constellation + neural demapper) — Classical 64-QAM + Gray + APP (baseline)

64-QAM (initialisation) — learned

Under AWGN, APP demapping is already optimal, so the improvement cannot come from the receiver. It comes from the transmitter. The learned constellation moves away from the regular 64-QAM lattice and gives more power to the symbols that are most often confused.

5 Performance Analysis

Table 1 —— E_b/N_0 needed for a given bit error rate (dB, lower is better). 64,000 codewords per point. Each estimate is based on at least 30 observed block errors.

System	1e-2	1e-3	1e-4
AWGN, classical 64-QAM + APP	5.67	6.17	6.54
AWGN, learned (channel gradient)	**5.30**	**5.78**	**6.18**
AWGN, learned (RL, no channel model)	5.42	5.92	6.31
Rayleigh, classical 64-QAM + APP	11.26	13.30	14.79
Rayleigh, learned	11.25	13.19	14.94

- Rayleigh fading requires **5.9 to 8.8 dB** more than AWGN, while the choice of receiver changes the result by less than **0.4 dB**.
- On AWGN the learned system gains **0.37, 0.39 and 0.36 dB** at BER 1e-2, 1e-3 and 1e-4. APP demapping is already optimal here, so the gain comes from **constellation shaping**.
- With only two pilot symbols per block, leaving the channel-estimation term out of the effective noise understates it by **1.76 dB**.
- Under Rayleigh fading with estimated CSI we measured **no gain** (+0.01, +0.11 and −0.15 dB). The value at 1e-4 changed sign between two runs, so we treat it as noise.

6 Why AWGN-Trained Weights Fail on Fading

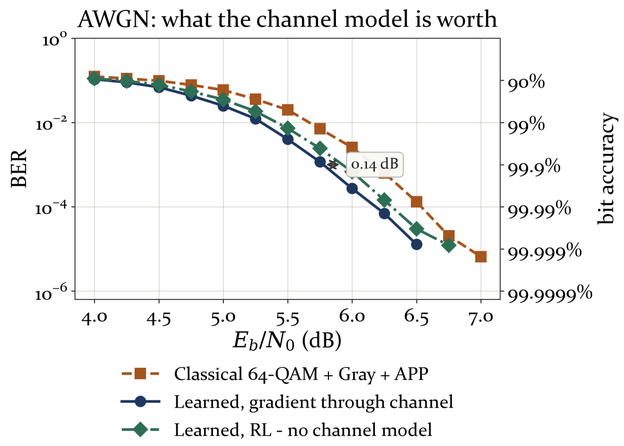
When the AWGN weights are used on a fading channel without retraining, the BER increases as the SNR increases. A real system should not behave this way, so we kept every weight fixed and changed one thing at a time:

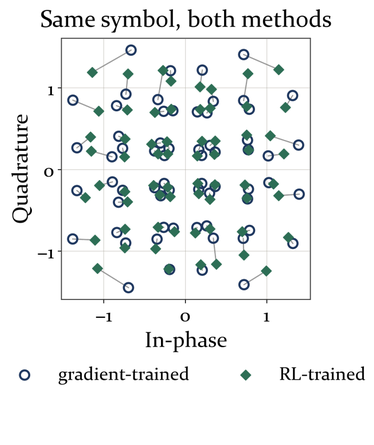
- Clamping the network outputs changed the BER by less than 4%, and the curve still increased.
- Removing the symbols in deep fades made the BER worse above 10 dB.
- Clamping the noise value passed to the demapper into the range it saw during training **restored a decreasing curve**, without changing any weight.

The cause is on the input side. At 18 dB, **89%** of symbols have an effective noise below anything the network saw in training, and only 4.6% are in deep fades. Retraining gives BER **1e-3** at 13.2 dB and **1e-4** at 14.9 dB.

7 Training Without a Channel Model

All the results above used backpropagation through a differentiable channel model. A deployed system does not have one. Here the receiver is trained as usual, but its gradient does not reach the channel. The transmitter perturbs its own symbols, receives one scalar loss value per codeword, and updates the constellation from that value [4]. The channel is only sampled.





- Removing the channel model costs **0.12, 0.14 and 0.13 dB** at BER 1e-2, 1e-3 and 1e-4.
- The reinforcement-learned system is still **0.23 to 0.25 dB** better than classical 64-QAM.
- The two training methods place each symbol at almost the same position, so reinforcement learning converges to a similar constellation.

8 Conclusion

On AWGN, end-to-end learning gives a gain of **0.37 dB**. Under Rayleigh fading with estimated CSI, we measured no gain. So the case for an AI-native transceiver is not accuracy. It is that the transmitter can be trained **with no channel model**, at a cost of **0.12 to 0.14 dB**.

9 Next Steps

- **Deploy on real hardware.** Embed the trained transceiver in a real device, such as a software-defined radio, and measure whether the gain still appears outside simulation.
- **Meta-learning for fast adaptation** [3]. The section above shows that the receiver fails when its input distribution changes.
- **Extend to OFDM and multipath** [2], where a convolutional demapper can use the correlation between neighbouring subcarriers.
- **Replace zero-forcing with MMSE equalisation**, which keeps the effective noise bounded.

Seeds are fixed, and every number is measured in the accompanying notebook. Sionna [5] and PyTorch.
Scope: flat block fading only, with no multipath, Doppler or OFDM.

References

[1] T. O'Shea and J. Hoydis, "An Introduction to Deep Learning for the Physical Layer," arXiv:1702.00832, 2017.

[2] H. Ye, G. Y. Li and B.-H. F. Juang, "Power of Deep Learning for Channel Estimation and Signal Detection in OFDM," arXiv:1708.08514, 2017.

[3] S. Smeka J. et al., "Accelerating Meta-Learning with the Enhanced Reptile Algorithm for Rapid Adaptation in Neural Networks," SCSE, IEEE, 2025.

[4] N. C. Luong et al., "Applications of Deep Reinforcement Learning in Communications and Networking: A Survey," IEEE Commun. Surveys Tuts., vol. 21, no. 4, 2019.

[5] NVIDIA Sionna, autoencoder tutorial notebook, which this work extends.